Frontend – Assignment 1

Q1. HTTP Request–Response Cycle

The HTTP Request–Response Cycle is the process by which a client (browser) communicates with a web server to fetch a resource (webpage, image, data, etc.) using the HTTP protocol.

Major Steps

1. **URL Entry / Trigger** – The user types a URL in the browser or clicks a link/submits a form. This initiates a request for a resource.
2. **DNS Resolution** – The browser translates the human-readable domain name (e.g., www.example.com) into an IP address using DNS (Domain Name System).
3. **TCP Connection Setup** – The browser establishes a TCP connection with the server using a three-way handshake (SYN, SYN-ACK, ACK). If HTTPS is used, an additional TLS/SSL handshake occurs for secure encryption.
4. **HTTP Request** – The browser sends an HTTP request to the server. This includes:
    - **Method** (GET, POST, PUT, DELETE, etc.)
    - **URL/Path** of the resource
    - **Headers** (User-Agent, Accept, Cookies, etc.)
    - **Body** (for POST/PUT requests, containing form data or JSON)
5. **Server Processing** – The web server receives the request, processes it (may involve routing, querying a database, running backend logic), and prepares a response.
6. **HTTP Response** – The server sends back an HTTP response containing:
    - **Status Code** (200 OK, 404 Not Found, 500 Internal Server Error, etc.)
    - **Headers** (Content-Type, Content-Length, etc.)
    - **Body** (HTML, JSON, image data, etc.)
7. **Rendering** – The browser receives the response and renders the content (parses HTML, applies CSS, executes JavaScript) to display the final page to the user.
8. **Connection Close/Reuse** – The TCP connection is either closed or kept alive (via Keep-Alive) for further requests, depending on HTTP version and headers.

Q2. Introduction to Web Technologies

1. HTML (HyperText Markup Language) Purpose: Provides the basic structure and content of a web page. Key Features: Uses tags/elements to define headings, paragraphs, links, images, tables, forms; platform-independent; forms the skeleton on which CSS and JavaScript operate.

2. CSS (Cascading Style Sheets) Purpose: Controls the visual presentation and layout of HTML elements. Key Features: Separates content from design; supports selectors, box model, flexbox/grid layouts, animations, responsive design via media queries.

3. JavaScript Purpose: Adds interactivity and dynamic behavior to web pages. Key Features: Client-side scripting language; manipulates the DOM, handles events, performs form validation, enables asynchronous operations (AJAX/Fetch), supports OOP and functional paradigms.

4. Bootstrap Purpose: A CSS framework used to build responsive, mobile-first websites quickly. Key Features: Pre-built grid system, ready-made UI components (navbars, buttons, modals, cards), responsive utilities, consistent cross-browser styling.

5. jQuery Purpose: A JavaScript library that simplifies DOM manipulation, event handling, and AJAX calls. Key Features: Write-less-do-more syntax ($() selector), cross-browser compatibility, simplified animations and effects, extensive plugin ecosystem.

6. React.js Purpose: A JavaScript library for building fast, component-based user interfaces (mainly SPAs - Single Page Applications). Key Features: Component-based architecture, Virtual DOM for efficient rendering, unidirectional data flow, JSX syntax, reusable UI components, strong ecosystem (React Router, Redux, etc.).

Q3. Client-Side vs. Server-Side Technologies

Client-Side Technologies run in the user's browser and are responsible for what the user sees and interacts with directly.

- Examples: HTML, CSS, JavaScript, React.js, Bootstrap, jQuery.
- Execution happens on the user's device.
- Used for UI rendering, interactivity, form validation, animations.

Server-Side Technologies run on the web server and handle backend logic such as processing requests, interacting with databases, and generating dynamic content before sending it to the client.

- Examples: Node.js, PHP, Python (Django/Flask), Java (Spring), ASP.NET.
- Execution happens on the server.
- Used for authentication, database operations, business logic, session management.

Key Differences

Basis	**Client-Side**	**Server-Side**
Execution Location	Browser (user's device)	Web server
Visibility	Code visible to users (view source)	Code hidden from users
Examples	HTML, CSS, JS, React	Node.js, PHP, Django
Dependency	Depends on browser capability	Depends on server resources

Basis	Client-Side	Server-Side
Security	Less secure (client can modify)	More secure (controlled access)
Use Case	UI, interactivity, presentation	Data processing, DB access, authentication

---

Q4. Block-Level Elements in HTML

Block-level elements are HTML elements that start on a new line and take up the full width available (by default), pushing subsequent content to the next line.

Characteristics

- Always starts on a new line.
- Occupies the full width of its parent container by default.
- Can contain other block-level and inline elements.
- Can have width, height, margin, and padding applied to all four sides.

Examples

1. – A generic container for grouping content.
2. <p> – Defines a paragraph.
3. <h1> to <h6> – Heading elements.
4. , <ol>, <li> – List elements.
5. <table> – Table element.

Q5. Semantic HTML Elements

Semantic elements are HTML tags that clearly describe their meaning and purpose to both the browser and the developer, rather than just defining a generic container.

Importance

- **Improved Accessibility** – Screen readers can better interpret page structure for visually impaired users.
- **SEO Benefits** – Search engines can better understand and index page content.
- **Code Readability & Maintainability** – Developers can understand the structure without relying on class/id names.
- **Consistent Structure** – Encourages a standardized way of organizing web pages.

Examples

- <header> – Defines introductory content or navigation links.
- <nav> – Defines a block of navigation links.

- <article> – Defines independent, self-contained content.
- <section> – Defines a thematic grouping of content.
- <footer> – Defines footer content (copyright, contact info).
- <aside> – Defines content aside from the main content (sidebar).
- <main> – Defines the main content of the document.

(Compare to non-semantic elements like and , which convey no meaning about their content.)

Q6. Creating a Table Using HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
 <title>Employee Table</title>
 <style>
  table {
   border-collapse: collapse;
   width: 60%;
   margin: 20px auto;
  }
  th, td {
   border: 1px solid #333;
   padding: 8px 12px;
   text-align: center;
  }
  th {
   background-color: #f2f2f2;
  }
 </style>
</head>
<body>
```

```
<table>
  <thead>
    <tr>
      <th>Sr. No.</th>
      <th>Name</th>
      <th>Designation</th>
      <th>Department</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>1</td>
      <td>John</td>
      <td>Manager</td>
      <td>Sales</td>
    </tr>
    <tr>
      <td>2</td>
      <td>Smith</td>
      <td>Sr. Manager</td>
      <td>Marketing</td>
    </tr>
    <tr>
      <td>3</td>
      <td>Jane</td>
      <td>Manager</td>
      <td>HR</td>
    </tr>
  </tbody>
</table>
```

```
</body>
</html>
```

Q7. Creating a Nested List Using HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Nested List</title>
</head>
<body>

 <ul>
   <li>River
     <ul>
       <li>Ganga</li>
       <li>Yamuna</li>
       <li>Brahmaputra</li>
       <li>Narmada</li>
     </ul>
   </li>
 </ul>

</body>
</html>
```

This uses an **unordered list (<ul>)** for "River" as the parent item, with a **nested unordered list** inside it for the sub-items (Ganga, Yamuna, Brahmaputra, Narmada), since these are unordered/non-sequential items.

Q8. Types of CSS

There are three main ways to apply CSS to an HTML document:

1. Inline CSS Applied directly to an individual HTML element using the style attribute.

```
<p style="color: blue; font-size: 16px;">This is inline CSS.</p>
```

- Highest specificity, but hard to maintain and reuse.

2. Internal CSS (Embedded CSS) Written within a <style> tag inside the <head> section of the HTML document. Applies to the whole page.

```
<head>
 <style>
  p {
   color: green;
   font-size: 16px;
  }
 </style>
</head>
```

- Useful for single-page styling, but not reusable across multiple pages.

3. External CSS Written in a separate .css file and linked to the HTML document using the <link> tag.

```
<head>
 <link rel="stylesheet" href="style.css">
</head>
/* style.css */
p {
 color: red;
 font-size: 16px;
}
```

- Best practice: promotes reusability, separation of concerns, and easier maintenance across multiple pages.

Q9. Practical Understanding: HTML vs CSS vs JavaScript

Technology	Role	Contribution
HTML	Structure	Defines the content and structure of the web page (headings, paragraphs, images, forms, links). It is the skeleton of the webpage.
CSS	Presentation	Controls how the content looks — colors, fonts, layout, spacing, responsiveness. It is the skin/styling of the webpage.

Technology	Role	Contribution
JavaScript	Behavior	Adds interactivity and dynamic functionality — form validation, event handling, animations, fetching data from servers. It is the brain/behavior of the webpage.

Analogy: If a webpage were a house — HTML is the structure (walls, rooms), CSS is the interior design (paint, furniture arrangement), and JavaScript is the electricity/automation (lights turning on, doors opening automatically).

Q10. Short Answers

a. Role of Bootstrap in web development Bootstrap is a front-end CSS framework that helps developers build responsive, mobile-first websites quickly using a pre-built grid system and ready-made UI components (buttons, forms, navbars, modals, cards). It ensures consistent design across browsers and devices, significantly reducing development time compared to writing custom CSS from scratch.

b. What is jQuery and why was it widely used? jQuery is a lightweight JavaScript library that simplifies HTML DOM manipulation, event handling, animation, and AJAX interactions. It was widely used because:

- It simplified complex JavaScript code into simple, one-line function calls.
- It resolved cross-browser compatibility issues (older browsers behaved differently with plain JS).
- It made AJAX calls and DOM traversal much easier.
- It had a huge plugin ecosystem for adding functionality quickly.

c. What is React.js and what problem does it solve? React.js is a JavaScript library developed by Facebook (Meta) for building user interfaces, particularly single-page applications (SPAs), using a component-based architecture.

It solves the problem of **inefficient UI updates and complex state management** in traditional web development:

- It uses a **Virtual DOM** to efficiently update only the parts of the actual DOM that change, improving performance.
- It promotes **reusable, modular components**, making large applications easier to build and maintain.
- It enables a **declarative UI approach** — developers describe what the UI should look like for a given state, and React handles the rendering, rather than manually manipulating the DOM step by step.